PeerChat: Secure Decentralized P2P Messaging System

---

1. Introduction

PeerChat is a decentralized peer-to-peer messaging platform built in Python. It enables direct, encrypted communication between users without relying on a centralized backend. A core principle of PeerChat is that **identity is cryptographically derived from RSA keys, ensuring that your username is uniquely yours.** Each node functions as both a client and a server, fostering a resilient, private, and scalable communication environment.

2. Capabilities

The PeerChat system has undergone significant enhancements, now offering a more comprehensive and refined set of features:

- **Context-Aware Filtering:** Private messages are distinctly isolated from Global Chat via a dual-pane UI, providing a focused communication experience.
- **Automatic Peer Discovery:** Dynamic network expansion is achieved through robust bootstrap peers, ensuring seamless connectivity.
- **RSA Identity Verification:** Secure challenge-response authentication is employed for cryptographic identity verification, enhancing trust and security.
- **Modernized GUI:** A polished PyQt6 interface features a dark mode and intuitive sidebar navigation, significantly improving the user experience.
- **Local Persistence:** Full message history is stored and retrieved efficiently via an integrated SQLite database.

- **File Attachments (New):** Direct peer-to-peer file sharing is now supported with an intuitive interface and automated local caching, enabling rich media exchange.

3. Detailed Features and Functions

3.1 Networking & Data Transfer

The networking layer is the backbone of PeerChat, ensuring dynamic, reliable, and secure peer-to-peer communication, now with robust file transfer capabilities.

Feature	Description
Fully Decentralized Architecture	Each peer operates autonomously, eliminating single points of failure and enhancing network resilience.
TCP Socket Communication	Utilizes TCP for reliable, ordered, and error-checked data transmission, now with newline-delimited JSON streaming for efficient packet handling.
Robust File Streaming	Binary files are automatically serialized using Base64 ASCII strings and optimized with buffered socket streams (1 MB allocation blocks) to allow seamless cross-platform delivery without data corruption or packet dropping.
Automatic Peer Discovery	Peers automatically find and connect to other nodes through bootstrap peers and dynamic peer list exchanges, ensuring continuous network expansion.
Multi-peer Communication	Supports simultaneous communication with multiple connected peers, facilitated by continuous receive loops for real-time message processing.
Connectivity Note	Users must ensure their trusted bootstrap peers are online and reachable on their specified ports to join the network swarm successfully.

3.2 Security

Security in PeerChat is paramount, employing advanced cryptographic methods for authentication, identity management, and secure key handling.

Feature	Description
RSA Public/Private Keys	Identity is cryptographically proven via RSA public/private key pairs, ensuring strong authentication.
Cryptographic Peer IDs	Peer IDs are generated as a hash of your RSA Public Key (`Username-Hash`), preventing identity spoofing and ensuring unique identification.
Challenge-Response Flow	A robust challenge-response mechanism prevents identity theft by requiring peers to sign a random challenge, without ever transmitting private keys or passwords.
Key Isolation	All sensitive `.pem` files (private and public keys) are securely stored in a dedicated `keys/` directory, organized by username, enhancing security and organization.

3.3 Messaging & Sharing

The messaging and sharing functions have been significantly enhanced to provide more organized, flexible, and rich communication options, including file attachments.

Feature	Description
Direct Private Messaging	Users can target specific peers for private, one-on-one conversations, which are filtered and displayed contextually in the UI.
Global Broadcast	Ability to send messages to all currently connected peers in the network swarm simultaneously.
File Attachments	A WhatsApp-style file sending interface is natively integrated next to the input message bar, allowing users to easily share files.
Automatic Downloads	Received attachments are automatically buffered, collision-checked to prevent filename overwriting, and stored securely in a local `downloads/` folder.
JSON Packet Protocol	Messages are encapsulated in a structured JSON format, used for identity, challenges, chat, and robust file segment mapping, ensuring extensible data exchange.
Persistence	All text messages and file-transfer reference markers are saved locally to `chat_history.db` , ensuring a complete and retrievable history.

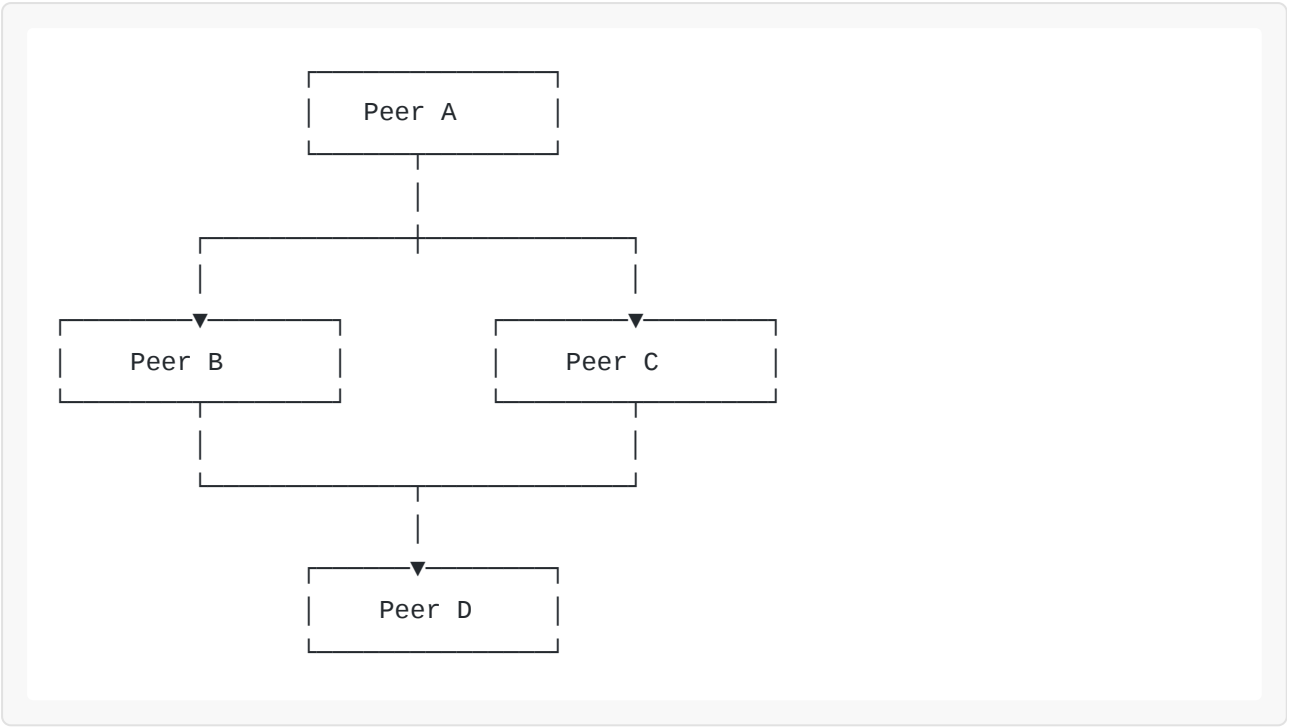
3.4 Graphical User Interface (GUI)

The PyQt6-based GUI has been further modernized to offer a more intuitive, aesthetically pleasing, and functional user experience, especially with the new file attachment features.

Feature	Description
Modern PyQt6 Interface	A sleek, dark-themed interface with a sidebar for peer selection and a clean chat area.
Attachment Button	A dedicated paperclip attachment icon (📎) is integrated, mapped directly to your operating system's native file explorer for easy file selection.
Enhanced Log Rendering	Context-aware UI highlighting dynamically wraps file attachments in custom stylized blocks to differentiate them from standard text messages, improving readability.
Identity Header	A clear display of your unique cryptographic Peer ID at the top right of the interface, reinforcing identity and trust.
Contextual History	Automatic message retrieval from SQLite based on the currently selected peer in the sidebar, providing a focused and relevant chat experience.

4. Architecture Overview

PeerChat maintains its decentralized model where each node is self-sufficient. The high-level architecture remains a mesh network, facilitating direct peer-to-peer connections:



Each peer node integrates several key functional components, now with explicit support for file handling:

- **Listener Server & Outgoing Client Connector:** Manages both incoming and outgoing network connections.
- **Message Handler & Peer Discovery Engine:** Processes messages and dynamically manages the list of active peers.
- **Authentication Layer (RSA):** Handles cryptographic authentication using RSA key pairs.
- **Local SQLite Database, Buffered Downloader, & Modern PyQt6 Frontend:** Provides persistent storage for chat history, robust handling of file downloads, and a user-friendly graphical interface.

5. Core Flows

5.1 Authentication Flow

PeerChat employs a robust cryptographic authentication process to verify peer identities without transmitting sensitive information:

1. **Connect:** Two peers establish a TCP connection.

2. **Exchange Public Keys (Identity Packet):** Peers exchange their RSA public keys encapsulated in an Identity Packet.
3. **Generate Random Challenge:** One peer generates a unique, random challenge string.
4. **Sign Challenge:** The peer receiving the challenge signs it using its private key.
5. **Verify Signature:** The challenging peer verifies the signature using the other peer's public key.
6. **Authenticated Communication Begins:** Upon successful verification, secure communication can proceed.

This flow ensures that peers can cryptographically prove their identity, making the system resistant to impersonation and replay attacks.

5.2 Peer Discovery Flow

The network expands dynamically through a multi-stage peer discovery process, ensuring robust and decentralized network growth.

1. **Bootstrap Peer:** A new peer initially connects to a pre-configured bootstrap peer.
2. **Request Peer List:** The new peer requests a list of known active peers from the bootstrap peer.
3. **Receive Discovered Peers:** The bootstrap peer responds with a list of IP addresses and ports of other peers.
4. **Auto-connect:** The new peer automatically attempts to connect to the discovered peers.
5. **Expand Network Graph:** As connections are established, peers continuously exchange their known peer lists, allowing the network to grow organically and dynamically.

6. Message Protocol

All inter-peer communication is facilitated using JSON-formatted packets, categorized by their `type` field. The protocol now explicitly supports direct, global, and file transfer messaging.

6.1 Chat Packet (Direct/Global)

Used for transmitting chat messages between peers, with a clear distinction for direct vs. global messages.

```
{
    "type": "chat",
    "data": {
        "sender": "Alice-a1b2c3d4",
        "recipient": "Bob-e5f6g7h8",
        "message": "Hello, Bob!"
    }
}
```

If `recipient` is null, the message is treated as a Global Broadcast.

6.2 File Transfer Packet (New)

Used for transmitting file attachments between peers.

```
{
    "type": "file_transfer",
    "data": {
        "sender": "Alice-a1b2c3d4",
        "recipient": "Bob-e5f6g7h8",
        "file_name": "document.pdf",
        "payload":
"JVBERi0xLjQKJbXtr90KMSAwIG9iagogIDw8IC9UeXBlIC9DYXRhbG9nCiAgICAvUGFn..."
    }
}
```

6.3 Identity Packet

Used during the initial handshake to exchange peer identification and public keys.

```
{
    "type": "identity",
    "data": {
        "peer_id": "5001",
        "public_key": "PEM_STRING"
    }
}
```

6.4 Challenge Packet

Used in the authentication flow to send a cryptographic challenge.

```
{
    "type": "challenge",
    "data": {
        "challenge": "ABCD1234"
    }
}
```

7. Project Structure and Module Overview

The PeerChat project has an updated and more organized folder structure, enhancing modularity and maintainability, with new additions for file handling.

```
peerchat/
│
├── main.py             # Entry point (Handles CLI arguments for
Port/Username)
├── config.py           # Runtime settings (Username, Peer ID, Port)
│
├── keys/               # Secure storage for RSA keys
│   ├── Alice_private.pem
│   └── Alice_public.pem
│
├── downloads/          # Auto-created folder for received file attachments
│   └── shared_image.png
│
├── gui/
│   ├── app.py          # Start GUI
│   ├── chat_window.py  # Attachment action trigger, UI rendering, &
Filtered logic
│   └── signals.py      # Event bus (3-part signaling: Sender, Msg,
Recipient)
│
├── network/
│   ├── protocol.py     # Packet creation/parsing
│   ├── server.py       # Handshake, Buffered File Writer, Routing & Signal
Emission
│   ├── client.py       # Connection, Base64 File Serializer & Transmission
logic
│   └── discovery.py    # Global peer registry
│
├── security/
│   ├── keys.py         # Public & private key generation
│   └── crypto.py       # Hashing, and filename management
│
└── storage/
    └── database.py     # SQLite history (get_history, save_message)
```

7.1 `network/` Module

This module is responsible for all aspects of inter-peer communication, including socket management, peer discovery, packet handling, and now robust file transfer.

- `server.py`: Manages incoming connections, handshake verification, buffered file writing, message routing, and signal emission for GUI updates.

- `client.py`: Handles outgoing connections, Base64 file serialization, and transmission logic.
- `protocol.py`: Defines the structure and parsing logic for JSON-based communication packets, including new file transfer types.
- `discovery.py`: Maintains the list of authenticated `peer_ids` for the GUI and manages peer registry.

7.2 `security/` Module

Dedicated to cryptographic operations, primarily focusing on peer authentication, key management, and now filename management for secure downloads.

- `keys.py`: Manages RSA key generation, loading, and enforces the dedicated `keys/` directory for storage.
- `crypto.py`: Provides functionalities for creating cryptographic signatures, verifying them, and now includes hashing and filename management for received files.

7.3 `storage/` Module

Handles the local persistence of application data, including chat history and file transfer references.

- `database.py`: Manages the SQLite database for storing chat messages and peer information, including `get_history` and `save_message` functions.

7.4 `gui/` Module

Contains all components related to the PyQt6 graphical user interface, now with enhanced features for file attachments and improved event handling.

- `app.py`: The main entry point for launching the GUI application.
- `chat_window.py`: Implements the primary chat interface, now with attachment action triggers, UI rendering, and filtered logic for private and global chats.
- `signals.py`: Defines custom signals for thread-safe communication between background network processes and the GUI event loop, now supporting 3-part signaling (Sender, Message, Recipient).

8. Running PeerChat

8.1 Install Dependencies

Install required Python packages:

```
pip install pyqt6 cryptography
```

8.2 Running Multiple Instances

To run peers locally, provide the **Port** and **Username** as command-line arguments. The system will automatically generate or load RSA keys for that username and store them in the `keys/` directory.

Terminal 1 (Alice):

```
python main.py 9000 Alice
```

Terminal 2 (Bob):

```
python main.py 9001 Bob
```

Terminal 3 (Charlie):

```
python main.py 9002 Charlie
```

9. Planned Improvements

PeerChat has a clear roadmap for future enhancements to further strengthen its capabilities and user experience.

- **E2EE Messaging:** Full implementation of End-to-End Encryption for message payloads and file buffers natively with RSA Public Keys/AES-GCM before transport serialization.
- **NAT Traversal:** Advanced techniques like UDP hole punching and STUN support to enable seamless over-the-internet P2P swarming.
- **Advanced UI:** Development of sophisticated UI elements such as unread message indicators and visual file transfer progress bars.

10. License

This project is licensed under the MIT License. Refer to the `LICENSE` file for complete details.